

Project 3

Goal

Gain experience in quickly learning a new programming language.

GitHub notes

Please see [GitHub notes](#) for more information on cloning, committing, and pushing repositories.

Assignment

Create a program that solves Sudoku in a language that is new to you.

At this point in the class, you have programmed a Sudoku solver in C++/Java and Racket, and have seen a solution in Prolog. You are now to write a Sudoku solver in a language that is new to you. The languages you may choose from are:

- Elixir
- Erlang
- F#
- Go
- Haskell
- Rust
- Scala**

** Using Java packages **prohibited**

In all cases, you can only use constructs based on the default installation of the language framework (i.e., do not use any special libraries that must be separately installed from the default installation).

Program features

Your program should implement the following features as seen in the previous two projects. Additional requirements are also included in this project. **Make sure to follow all the requirements for each of the features.** Failure to do so will result in points being deducted.

- **Read** puzzle from a file.
- **Print** the initial puzzle and solved puzzle (and a no solution message if unsolvable).
- Use the language's testing framework to **run tests**.
- Configure GitHub Actions to run the tests on **online**.

Reading from a file

Your program should **prompt** the user for a file name and **load** the puzzle from that file. Checking that a file with the given file name actually exists is not required. Text files containing the three puzzles from the previous projects should be included (i.e., `sudoku-test1.txt` , `sudoku-test2.txt` , and `sudoku-impossible.txt`).

When your program runs, a prompt should be displayed that includes instructions on how to enter name of the puzzle text file (just like in Project 1). This should include whether we need to include the "txt" folder in the input or not (don't leave us guessing). As a related feature, if we just press <Enter>, then the `sudoku-test1.txt` puzzle text file should be used.

Note that the previous projects imported a puzzle in two different ways. In the Project 1 (C++/Java), the contents of the text file were read one token at a time and stored in some data structure. In Project 2 (Racket), the text file contained the actual code for the data structure in textual or serialized form (i.e., a list of lists) and built-in functions were utilized to import the textual version of the list of lists into the program. You may implement the reading of the text files either way.

Printing the puzzle

Just as in previous projects, both the **initial** puzzle and the **solved** puzzle should be **printed**. The printed puzzle should include borders just as in previous projects (i.e., `+-----+-----+-----+` at the top and bottom of the puzzle and after each submatrix, and `|` on the left and right of the puzzle and also after each submatrix).

If the puzzle is not solvable, the message "**This puzzle has no solution.**" message should be printed.

Timing information is not necessary.

Testing the solver

All the languages listed above have a **testing framework** that can be used to confirm that the solver worked correctly. If you look at the tests from the previous projects, the general process is to load the unsolved puzzle and the solved puzzle from the text files. Then run the solver and compare or *assert* that the result generated by the solver is equal to the solved puzzle. The three test puzzles: `sudoku-test1.txt` , `sudoku-test2.txt` , and `sudoku-impossible.txt` should be included. You're welcome to test your solution on other puzzles, but your final submission should only include tests on the three puzzles from the previous projects.

The testing must be done within the testing framework of the language, which allows for the test cases to be invoked via a special command separate from the running of the program.

When testing your solved puzzle, you must compare the solved puzzle with the solution in some way. You can't just assert that some function that is tasked to solve the puzzle returns `true` and that signifies that the puzzle was solved correctly.

You should only perform tests on the three puzzles (i.e., `sudoku-test1.txt` , `sudoku-test2.txt` , and `sudoku-impossible.txt`). Do not include testing on the equals method and the printout of the puzzle (such as was done in Project 1). Also, do not include testing of whether the functions that check if a value can be placed in a position on the puzzle are correct (i.e., the first 20 test cases in Project 2).

Testing on [GitHub.com](https://github.com)

Your Project 3 GitHub repository should be configured so that the test cases can also be run on [GitHub.com](https://github.com). This requires configuring a **GitHub Actions** workflow. [A description of GitHub Actions workflow setup in the previous projects.](#)

If you setup the workflow correctly, once you push to [GitHub.com](https://github.com) and all the tests pass, a green checkmark `:white_check_mark:` will be displayed beside the commit information.

IMPORTANT

The personal access token that you are using to access your GitHub repository must have the "workflow" scope included. This scope should be included when you followed the instructions for Homework 0. To confirm this on [GitHub.com](https://github.com), click on your avatar at the top right and select **Settings | Developer Settings | Personal access tokens | Tokens (classic)**. Click on the token that you are using and make sure the **workflow** scope is checked. If it isn't, then select it and click **Update token** at the bottom of the page. You may also create a new personal access token with the "repo" and "workflow" scope checked.

Project submission

For submission, the individual or team repository for Project 3 on GitHub should contain the following files:

- **All source code files** – Make sure to add header comments to the top of all source files. Placing the files in a `src` folder is optional. Some languages allow for both the implementation code and test code to be in the same file. While this is not ideal, you may place all the code in one file.

Also, include any text-based configuration files needed to compile and run your program.

- **Puzzle text files** – Include the two solvable puzzles and one unsolvable puzzle from past projects (i.e., they must be exactly the puzzle instances from `sudoku-test1.txt`, `sudoku-test2.txt`, and `sudoku-impossible.txt` that were provided in the previous two projects). Placing the files in a `txt` folder is optional.
- Update the [project3.md](#) file and include all required information outlined in the file, which include:
 - **Names** of individual or team members.
 - **Instructions** on how to access an appropriate **compiler or interpreter** for your solver. If you found a website with appropriate installation instructions, you can just include the website's URL. You don't need to provide installation instructions for all operating systems. You can just provide instructions for the operating system that you used for the project.
 - **Instructions on how to use and test** your program on the command prompt. We expect to run your program on the command prompt and **not** through an IDE interface. Provide us with the specific command that needs to be typed to run your program. Also, tell us if we need to navigate to a specific folder first before typing the command. The same goes for how to test your program using the testing framework. Provide us with the specific command to run the test cases on your program.
 - **Answer** the questions listed in the file.

Only the files originally in the repository and any additional files specified above and the folders they are located in should be included in your repository on [GitHub.com](https://github.com). There should be no other files or folders such as binary files and operating system files. Take a look at Git's `.gitignore` file to see how to *ignore* these files when committing (and

pushing). The files can reside in the repository on your local computer, but it should not be included in the repository on [GitHub.com](https://github.com).

Markdown markup language

The [project3.md](#) file is a *Markdown* file, which allows you to use simple syntax to format the text in the file. You can open and edit [project3.md](#) in your favorite text editor. [More information on the Markdown syntax](#).

Your [project3.md](#) file is expected to be reasonably organized and neat. Look at the [readme.md](#) files from previous assignments for example usage of Markdown style. The text should not be messy and/or disorganized. Also, the text should not look as though it was just copied-and-pasted without any formatting.

Participation form for team projects

Important: For those who are on a team, each team member must complete the [Google form](#) by the project due date on **Wednesday (04/22)** discussing how much each team member contributed to the project. Those working individually do not need to complete the form.

Your submission to the form stating your personal opinion will not be shared with other team members. You should state a percentage of contribution by each team member (should total 100%) and you should provide additional commentary describing each team member's contribution to the overall effort. Optionally, you can provide information on any special circumstances you want me to be aware of. Note that we will not look at how active each team member is on the GitHub repository (e.g., committing code). Team participation will be based on the form above.

Even if different team members work on different parts of the project, each team member should have a general understanding of all the aspects of the project.

Style guidelines

- **Structure:** You should not place the entire solution in one function/method. Make sure you modularize your code (i.e., place code for sub-tasks in different functions/methods).
- **Line lengths:** No lines should exceed 100 columns.
- **Indentation:** Use proper and consistent indentation.
- **Whitespace:** Functions/methods should be separated by at least one empty line.
- **Comments:** Use Javadoc-style comments to document all functions/methods. For your reference, below is an example of Javadoc-style comments for a method.

```
/**
 * Returns an Image object that can then be painted on the screen.
 *
 * @param url An absolute URL giving the base location of the image.
 * @param name The location of the image, relative to the url argument.
 * @return The image at the specified URL.
 */
```

You do not need to include Javadoc-style comments on functions/methods that are in any test files. That is, only provide Javadoc-style comments on functions/methods containing implementation code. However, header comments containing your name and email address must be placed on both implementation and test files.

Grading

In addition to deductions if the solver implementation is not correct, points will be deducted for the following issues:

- If any of the required program features are incorrect or missing.
- If the [project3.md](#) file is incomplete, messy, or disorganized.
- If the green checkmark associated to a GitHub Actions workflow is falsified. That is, if your repository displays a green checkmark, but the workflow did not concretely test the solver.
- Unnecessary files (i.e., files that were not required to be included) are in the repository.

Not completing the language selection and working individually or as a team form by the deadline will be deducted around 5%.

If you are encountering difficulties and need to change the language that you initially chose, you must contact the instructor about the change. Note that your score will be reduced by around 5%.

A team member who does not complete the team participation form by the deadline will be deducted around 5% from that team member's score. A team member who did not contribute much to the project will have their score reduced by around 10%.

Graduate students

You are required to select a second language and create a Sudoku solver with that language, too. For the second language, you **are not** allowed to join a team. That is, for one of your languages you can join a team, but for the other you must work individually. Of course, you may work on both individually, too. Please email your second language selection directly to the instructor. The due date for this second language is the same as the due date of the first language, which is on **Wednesday (04/22)**.

Final notes

- Ensure that your **name(s)** and **email address(es)** appear as header comments of files containing code (both implementation and test files).
- You must **push** your repository to [GitHub.com](#) **before** the deadline as **no late submissions** will be accepted.
- You should make sure that a push to GitHub will automatically **trigger** a build or builds. All builds **must pass** for the submission to be considered for full credit.
- CS 3270 TAs **are not** expected to know the languages in Project 3, so they **are not** obligated to provide assistance.